

Autoencoders and Regularisation

Vatsav Reddy

Under the Guidance of
Prof. Praveen Parchuri

International Institute of Information Technology, Hyderabad

June 9, 2026

Contents

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout
- 19 Recap: Training Deep Neural Networks
- 20 DCT and JPEG Image Compression
- 21 Summary

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)

What is an Autoencoder?

Definition

An **autoencoder** is a special feed-forward neural network that learns to copy its input to its output through a compressed intermediate representation. It performs two operations:

- **Encode:** map input $\mathbf{x}_i$ into a hidden representation $\mathbf{h}$.
- **Decode:** reconstruct the input from $\mathbf{h}$ to produce the output $\hat{\mathbf{x}}_i$.

The Two Maps

$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b}) \qquad \hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c})$$

with $\mathbf{x}_i \in \mathbb{R}^n$, $\mathbf{h} \in \mathbb{R}^d$, $W \in \mathbb{R}^{d \times n}$, $W^* \in \mathbb{R}^{n \times d}$. Here g is the encoder *nonlinearity* and f the decoder nonlinearity.

Goal

We want $\mathbf{h}$ to capture the *most important characteristics* of $\mathbf{x}_i$: everything required to reconstruct the original input. Training minimises a loss that forces $\hat{\mathbf{x}}_i \approx \mathbf{x}_i$.

Undercomplete Autoencoders

When $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$

The hidden layer is *smaller* than the input. If the network can still reconstruct $\mathbf{x}_i$ from $\mathbf{h}$, then $\mathbf{h}$ is a **loss-free encoding**: it has captured all the important characteristics of $\mathbf{x}_i$ in fewer dimensions.

The Analogy with PCA

This is exactly what PCA does. The bottleneck $\mathbf{h}$:

- gets rid of **noise**,
- discards **low-variance** directions,
- removes **correlated** / **redundant** dimensions.

We will later prove that a *linear* undercomplete autoencoder is mathematically equivalent to PCA.

Compression Intuition

Think of a 10-bit object compressed to a 4-bit code, then expanded back to 10 bits. If reconstruction succeeds, the 4-bit code retained all essential information — the rest was redundant.

Overcomplete Autoencoders

When $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$

The hidden layer is *at least as large* as the input. In this case the autoencoder can learn a **trivial identity encoding**: simply copy $\mathbf{x}_i$ into $\mathbf{h}$, then copy $\mathbf{h}$ into $\hat{\mathbf{x}}_i$.

Why the Identity Encoding is Useless

An identity mapping reconstructs perfectly but tells us *nothing* about the important characteristics of the data — no compression, no insight, no denoising. It is like copying a file rather than compressing it.

The Core Problem

There is *no reason* the network should choose to disentangle features rather than copy them. Left alone, it tends to learn the easy identity map. Hence we need **special methods** (regularisation) to prevent this kind of trivial representation — this motivates the entire second half of these notes.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function**
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Two Design Choices

What Must Be Chosen

Designing an autoencoder requires two decisions:

- 1 the decoder activation f (and encoder activation g),
- 2 the reconstruction **loss function**.

Both depend on the *type* of input data (binary vs. real-valued).

Binary Inputs

Suppose each $x_{ij} \in \{0, 1\}$ is a binary variable. Candidate decoders:

$$\hat{x}_i = \tanh(W^* \mathbf{h} + \mathbf{c}) \in [-1, 1], \quad \hat{x}_i = W^* \mathbf{h} + \mathbf{c} \in \mathbb{R}, \quad \hat{x}_i = \text{logistic}(W^* \mathbf{h} + \mathbf{c})$$

Since inputs lie in $\{0, 1\}$, the **logistic (sigmoid)** decoder is most apt — it naturally restricts outputs to $[0, 1]$ and can be read as a probability. The encoder g is also typically the sigmoid.

Real-Valued Inputs

For $\mathbf{x}_i \in \mathbb{R}^n$ (e.g. pixel intensities 0.25, 0.5, ...), logistic and tanh would wrongly restrict the output to $[0, 1]$ or $[-1, 1]$. The most apt decoder is **linear**: $\hat{x}_i = W^* \mathbf{h} + \mathbf{c}$, since the output should lie in $\mathbb{R}$. Again g is typically the sigmoid.

Loss Function: Real-Valued Inputs (Squared Error)

Objective

The objective of the autoencoder is to reconstruct $\hat{\mathbf{x}}_i$ as close to $\mathbf{x}_i$ as possible. For real-valued inputs we use **squared-error** loss:

$$\min_{W, W^*, \mathbf{b}, \mathbf{c}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2 = \min_{W, W^*, \mathbf{b}, \mathbf{c}} \frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{x}}_i - \mathbf{x}_i)^\top (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

This is just the average squared ℓ_2 distance between the reconstruction and the input over all m training points.

Training via Back-Propagation

The autoencoder is trained exactly like a regular feed-forward network using back-propagation. All we need is a formula for the gradient of the loss with respect to the output of the last layer. With $L(\theta) = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^\top (\hat{\mathbf{x}}_i - \mathbf{x}_i)$:

$$\frac{\partial L(\theta)}{\partial \hat{\mathbf{x}}_i} = \nabla_{\hat{\mathbf{x}}_i} (\hat{\mathbf{x}}_i - \mathbf{x}_i)^\top (\hat{\mathbf{x}}_i - \mathbf{x}_i) = 2(\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

From this seed gradient, the chain rule propagates derivatives back through decoder and encoder, exactly as in standard back-propagation.

Loss Function: Binary Inputs (Cross-Entropy)

Why a Probabilistic Loss?

With a sigmoid decoder the output $\sigma(\cdot) \in (0, 1)$ can be read as a *probability*: an output of 0.8 means the network is 80% confident the true value is 1, and 0.2 means 20% confident it is 1 (i.e. 80% confident it is 0). Whenever a probabilistic interpretation is possible, **cross-entropy** loss often works much better than squared loss.

Cross-Entropy Reconstruction Loss

$$\min -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n \left[x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}) \right]$$

derived from the general cross-entropy $-\sum_l p_l \log q_l$ with the true distribution $p = (1 - x_{ij}, x_{ij})$ and estimate $q = (1 - \hat{x}_{ij}, \hat{x}_{ij})$.

Which $\hat{x}_{ij}$ Minimises This?

Treating one term: if $x_{ij} = 1$ the loss is $-\log \hat{x}_{ij}$, minimised by $\hat{x}_{ij} = 1$. If $x_{ij} = 0$ the loss is $-\log(1 - \hat{x}_{ij})$, minimised by $\hat{x}_{ij} = 0$. So the optimum is precisely $\hat{x}_{ij} = x_{ij}$ — the loss rewards matching the target. (With $h_{2j} = \hat{x}_{ij} = \sigma(a_{2j})$.)

Back-Propagation through the Cross-Entropy Loss

Seed Gradient at the Output

For a single output unit with target x_{ij} and reconstruction $\hat{x}_{ij} = \sigma(a_{2j})$, the loss $L(\theta) = -[x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij})]$ gives

$$\frac{\partial L(\theta)}{\partial \hat{x}_{ij}} = -\frac{x_{ij}}{\hat{x}_{ij}} + \frac{1 - x_{ij}}{1 - \hat{x}_{ij}}$$

and through the sigmoid pre-activation a_{2j} , $\frac{d\hat{x}_{ij}}{da_{2j}} = \sigma(a_{2j})(1 - \sigma(a_{2j}))$.

Chain Rule Assembles the Full Gradient

$$\frac{\partial L(\theta)}{\partial W} = \underbrace{\frac{\partial L(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2}}_{\text{reuse}} \frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}$$

The boxed quantity is computed once and *reused*; only the term that varies in the summation index j gets differentiated. This is the standard back-propagation recipe applied to the autoencoder.

Summary of Choices

Binary input $\Rightarrow$ sigmoid decoder + cross-entropy loss. **Real-valued input** $\Rightarrow$ linear decoder + squared-error loss. The encoder g is typically sigmoid in both cases. The hidden layer is the *bottleneck* that defines the learned representation.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders**
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)

Claim: A Linear Autoencoder is PCA

The Four Conditions

The *encoder* of an autoencoder is equivalent to PCA provided we:

- 1 use a **linear encoder** ($\mathbf{h} = W\mathbf{x}$, not $\sigma(W\mathbf{x})$),
- 2 use a **linear decoder** ($\hat{\mathbf{x}} = W^*\mathbf{h}$),
- 3 use a **squared-error** loss function,
- 4 **normalise** the inputs as below.

Normalisation (Centring + Scaling)

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \frac{1}{m} \sum_{k=1}^m x_{kj} \right)$$

The bracket subtracts the column (feature) mean, so the data has **zero mean** along each dimension j . Let X' be this zero-mean data matrix; then $X = \frac{1}{\sqrt{m}} X'$.

Why This Matters

With this scaling, $X^\top X = \frac{1}{m} (X')^\top X'$ is exactly the **covariance matrix** — and the covariance matrix is what PCA diagonalises. This is the bridge between the two methods.

Reformulating the Objective via the Frobenius Norm

Squared Error as a Matrix Norm

The reconstruction objective $\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (x_{ij} - \hat{x}_{ij})^2$ is precisely a **Frobenius norm**:

$$\min_{W, W^*} \|X - HW^*\|_F^2$$

where row i of H is the code $\mathbf{h}_i$ and $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$ is the square root of the sum of squared entries. (We ignore the biases here.)

Dimensions

With m points in $\mathbb{R}^n$ and code size k : $X \in \mathbb{R}^{m \times n}$, $H \in \mathbb{R}^{m \times k}$ (encoder output, $H = XW^\top$ for a linear encoder with $W \in \mathbb{R}^{k \times n}$), $W^* \in \mathbb{R}^{k \times n}$ (decoder). We seek the best rank- k factorisation HW^* of the data X .

Key Recognition

Writing $\hat{X} = HW^*$, the problem is $\min \|X - \hat{X}\|_F^2$ over rank- k matrices $\hat{X}$ — *exactly* the best low-rank approximation problem solved by SVD (Eckart–Young).

Solving with SVD

Optimal Low-Rank Solution

Let $X = U\Sigma V^\top$ be the SVD of the centred data. By the Eckart–Young theorem the optimal rank- k reconstruction is

$$\hat{X} = U_{:, \leq k} \Sigma_{k \times k} V_{:, \leq k}^\top$$

where $U_{:, \leq k}$ keeps the first k columns, etc.

Matching Variables

Comparing $\hat{X} = HW^*$ with $U_{:, \leq k} \Sigma_{k \times k} V_{:, \leq k}^\top$, one valid identification is

$$H = U_{:, \leq k} \Sigma_{k \times k}, \quad W^* = V_{:, \leq k}^\top.$$

We now show the corresponding **encoder** is linear and recover its weight matrix W .

Recovering the Encoder Weights W (so that $H = XW^\top$, here written $H = MX$ logic)

We use $X = U\Sigma V^\top$, $V^\top V = I$, $U^\top U = I$, the pseudo-inverse identity $(XX^\top)(XX^\top)^{-1} = I$, and the transpose rule $(ABC)^\top = C^\top B^\top A^\top$.

Deriving $H = X V_{\cdot, \leq k}$ (the Encoder is Linear)

Express H Using Only X

Start from $H = U_{\cdot, \leq k} \Sigma_{k \times k}$ but we do not want U explicitly — we want H as a linear map of X . Substitute $U_{\cdot, \leq k} \Sigma_{k \times k} = (XX^\top)(XX^\top)^{-1} U_{\cdot, \leq k} \Sigma_{k \times k}$ and $X = U \Sigma V^\top$:

$$\begin{aligned} H &= (XX^\top)(XX^\top)^{-1} U_{\cdot, \leq k} \Sigma_{k \times k} \\ &= (XV \Sigma^\top U^\top)(U \Sigma V^\top V \Sigma^\top U^\top)^{-1} U_{\cdot, \leq k} \Sigma_{k \times k} \\ &= XV \Sigma^\top U^\top (U \Sigma \Sigma^\top U^\top)^{-1} U_{\cdot, \leq k} \Sigma_{k \times k} \\ &= XV \Sigma^\top U^\top U (\Sigma \Sigma^\top)^{-1} U^\top U_{\cdot, \leq k} \Sigma_{k \times k} \\ &= XV \Sigma^\top (\Sigma \Sigma^\top)^{-1} \underbrace{U^\top U_{\cdot, \leq k}}_{I_{\cdot, \leq k}} \Sigma_{k \times k} \\ &= XV \Sigma^{-1} I_{\cdot, \leq k} \Sigma_{k \times k} = XV I_{\cdot, \leq k} = X V_{\cdot, \leq k}. \end{aligned}$$

Conclusion

$H = X V_{\cdot, \leq k}$ is a **linear transformation** of X . Therefore the optimal encoder is a *linear* encoder with weights $W = V_{\cdot, \leq k}$.

The Encoder Weights Equal the PCA Projection Matrix

Chain of Identities

- We have the encoder $W = V_{:, \leq k}$.
- From SVD: V consists of the **eigenvectors of $X^\top X$** .
- From PCA: the projection matrix P consists of the **eigenvectors of the covariance matrix**.
- With the normalisation $\hat{x}_{ij} = \frac{1}{\sqrt{m}}(x_{ij} - \frac{1}{m} \sum_k x_{kj})$, the matrix $X^\top X$ is the covariance matrix.

Hence Proved

The encoder matrix W of a linear autoencoder and the projection matrix P of PCA are computed from the eigenvectors of the same covariance matrix — so they are the **same**. A linear undercomplete autoencoder *is* PCA. □

Remember — the Four Conditions

Linear encoder, linear decoder, squared-error loss, and the $\frac{1}{\sqrt{m}}$ zero-mean normalisation. Drop any one (e.g. use a sigmoid encoder) and the equivalence no longer holds.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders**
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Why Regularisation is Needed

Overfitting and Capacity

More learnable parameters $\Rightarrow$ more overfitting $\Rightarrow$ less generalisation. An overcomplete autoencoder ($\dim(\mathbf{h}) \geq \dim(\mathbf{x})$) has many parameters and can simply learn to **copy** $\mathbf{x}_i \rightarrow \mathbf{h} \rightarrow \hat{\mathbf{x}}_i$.

Even Undercomplete Autoencoders Can Overfit

Consider a 100×100 image: $n \approx 10^4$ inputs with a code $k = 1000$. Although $k < n$, the model *still* has a very large number of parameters. High-dimensional inputs with much redundancy mean that even shrinking the input still leaves a large parameter count, so poor regularisation can occur even in the undercomplete case. It is a *more serious* problem for overcomplete autoencoders.

The Fix

To avoid poor generalisation we introduce **regularisation**. The simplest solution is to add an L_2 penalty to the objective:

$$\min_{\theta, W, W^*, \mathbf{b}, \mathbf{c}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2 + \boxed{\lambda \|\theta\|^2}$$

The penalty stops weights from growing; it does not give the model enough freedom to overfit / memorise training data, so we hope it generalises well on test data.

Weight Tying

Tie Encoder and Decoder Weights

A second trick: force the decoder weights to be the transpose of the encoder weights,

$$\boxed{W^* = W^\top} \quad (W \in \mathbb{R}^{k \times d}, W^* \in \mathbb{R}^{d \times k}).$$

This is enforced as a hard constraint: the same weight matrix is learned in such a way that it works at *both* the encoder and the decoder.

Why It Regularises

Tying **halves** the number of free parameters ($2dk \rightarrow dk$). Fewer parameters $\Rightarrow$ may generalise better. Geometrically, the encoder projects onto feature directions and the decoder reconstructs from those *same* directions; this reduces freedom and acts as regularisation. It is commonly used for autoencoders.

Implementation Note

L_2 regularisation is trivial to implement: it just adds a term λw (or αw) to the gradient $\partial L / \partial w$, and similarly for the other parameters.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders**
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)

The Denoising Idea

Corrupt the Input

A denoising autoencoder **corrupts** the input data using a probabilistic process $P(\tilde{x}_{ij} \mid x_{ij})$ *before* feeding it to the network. A simple corruption used in practice:

$$P(\tilde{x}_{ij} = 0 \mid x_{ij}) = q, \quad P(\tilde{x}_{ij} = x_{ij} \mid x_{ij}) = 1 - q.$$

With probability q each input is flipped to 0; with probability $1 - q$ it is retained as is.

The Objective Stays the Same

Crucially, the objective is still to reconstruct the *original, uncorrupted* $\mathbf{x}_i$:

$$\arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

The network receives $\tilde{\mathbf{x}}_i$ but is graded against $\mathbf{x}_i$.

Effect

This is a form of regularisation that makes the representation **more robust**.

Why Denoising Helps

The Network Can No Longer Just Copy

With corruption, copying $\tilde{\mathbf{x}}_i \rightarrow \mathbf{h} \rightarrow \hat{\mathbf{x}}_i$ no longer minimises the objective: a copied corrupted pixel will not match the clean target, incurring error.

It Must Learn Structure

Instead the model must learn to reconstruct a corrupted x_{ij} *correctly* by relying on its **interaction with other elements** of $\mathbf{x}_i$. For example, a missing pixel must be inferred from neighbouring pixels. It therefore has to capture the true characteristics of the data, not memorise it.

Train vs. Test

Because copying is penalised, the model can no longer get away by memorising the data — yet at test time (clean inputs) it still works, because it learned genuine structure during training. Denoising is thus a kind of regularisation.

Application: Handwritten Digit Recognition

Setup

MNIST-style digits: each image is $28 \times 28 = 784$ pixels, so $\mathbf{x}_i \in \mathbb{R}^{784}$, and labels are $0, \dots, 9$.

Basic Approach vs. Autoencoder Approach

- **Basic:** feed the 784 raw pixels directly to a classifier. Requires manual feature engineering and is hard.
- **Autoencoder approach:** first learn the important characteristics of the data with an autoencoder, obtaining a *better, smaller* representation $\mathbf{h} \in \mathbb{R}^d$ ($d < 784$). No feature engineering needed.

Pipeline

Train the autoencoder $\rightarrow$ reuse the encoder neural network to map each image to its code $\mathbf{h} \rightarrow$ feed the better representation to a multi-class SVM. Using a good learned representation makes the downstream classification far easier than feeding raw pixels. (Downside: less explainability.)

Visualising an Autoencoder: Neurons as Filters

A Neuron is a Filter

Each hidden neuron can be viewed as a **filter** that fires (gets maximally activated) for a certain configuration of the input. For the first hidden neuron (ignoring bias),

$$h_1 = \sigma(\mathbf{w}_1^\top \mathbf{x}_i),$$

where $\mathbf{w}_1$ is the trained weight vector connecting to that neuron.

Which Input Maximises a Neuron?

Training is done, so $\mathbf{w}_1$ is fixed. We ask: *what input $\mathbf{x}_i$ maximally fires this neuron?*

Assume inputs are normalised, $\|\mathbf{x}_i\| = 1$:

$$\max_{\mathbf{x}_i} \mathbf{w}_1^\top \mathbf{x}_i \quad \text{s.t.} \quad \|\mathbf{x}_i\|^2 = \mathbf{x}_i^\top \mathbf{x}_i = 1.$$

A dot product $\mathbf{w}_1^\top \mathbf{x}_i = \|\mathbf{w}_1\| \|\mathbf{x}_i\| \cos \theta$ is maximised when $\cos \theta = 1$, i.e. both vectors point in the **same direction**.

The Maximally-Activating Input

Solving the Constrained Maximisation

With the constraint $\|\mathbf{x}_i\| = 1$, Cauchy–Schwarz gives $\mathbf{w}_1^\top \mathbf{x}_i \leq \|\mathbf{w}_1\|$, with equality when $\mathbf{x}_i \propto \mathbf{w}_1$. Solving for $\mathbf{x}_i = c\mathbf{w}_1$ under $\|\mathbf{x}_i\| = 1$ gives $c = 1/\|\mathbf{w}_1\|$, so

$$\mathbf{x}_i = \frac{\mathbf{w}_1}{\sqrt{\mathbf{w}_1^\top \mathbf{w}_1}}$$

More generally, the inputs $\mathbf{x}_i = \frac{\mathbf{w}_l}{\sqrt{\mathbf{w}_l^\top \mathbf{w}_l}}$ cause hidden neurons $l = 1, \dots, k$ to maximally fire.

Reading the Filters

Plotting these maximally-activating images (reshaped to 28×28) reveals the patterns each neuron detects. For a vanilla autoencoder these resemble pen-strokes / parts of digits; comparing them across different autoencoders shows what each model has learned.

Denoising Filters: Pen-Stroke Detectors

Empirical Observation

The hidden neurons of a denoising autoencoder act like **pen-stroke detectors**. For example, a highlighted neuron may respond to a stroke in the back region that one would expect in a '0', '2', '3', '8' or '9'.

Effect of Noise Level

As the corruption noise q *increases*, the learned filters become **wider**. The reason: the neuron must rely on a larger set of adjacent pixels to feel confident about a stroke, since any single pixel may have been corrupted. A vanilla (no-noise) autoencoder does not learn many such interpretable patterns and relies more heavily on a few weights.

Gaussian Corruption

Besides flipping pixels to zero, another corruption process adds Gaussian noise to the input:

$$\tilde{x}_{ij} = x_{ij} + \mathcal{N}(0, 1).$$

Empirically, with Gaussian noise the AE hidden neurons behave like edge detectors, whereas a weight-decay-only filter does not capture anything special like edges.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders**
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)

Sparsity: Keep Most Neurons Inactive

Activation of a Sigmoid Neuron

A hidden neuron with sigmoid activation outputs a value in $(0, 1)$. We say it is **activated** when its output is close to 1 and **inactive** when close to 0. A *sparse* autoencoder tries to ensure each neuron is inactive most of the time.

Average Activation

The average activation of hidden unit l over the dataset is

$$\hat{\rho}_l = \frac{1}{m} \sum_{i=1}^m h(\mathbf{x}_i)_l.$$

If unit l is sparse (mostly inactive) then $\hat{\rho}_l \rightarrow 0$.

The Constraint

A sparse autoencoder uses a small **sparsity parameter** ρ (typically close to zero, e.g. 0.005) and enforces $\hat{\rho}_l = \rho$ for every hidden unit l . Intuition: if a neuron fires for *every* input it carries no discriminative power; forcing it to fire only for some inputs makes it capture a specific pattern. Any kind of regularisation restricts the freedom of parameters — overfitting happens because of too much freedom.

The Sparsity (KL-Divergence) Penalty

Adding a Term to the Objective

To enforce $\hat{\rho}_l = \rho$ we add the following regularisation term (a sum of KL divergences) to the loss:

$$\Omega(\theta) = \sum_{l=1}^k \rho \log \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_l}$$

The full regularised objective is

$$\widetilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

where $\mathcal{L}(\theta)$ is the usual squared-error / cross-entropy reconstruction loss.

Minimum of the Penalty

Ω reaches its minimum value (of 0) exactly when $\hat{\rho}_l = \rho$ for all l — it is the KL divergence between two Bernoulli distributions, which is 0 iff they coincide, and grows sharply as $\hat{\rho}_l$ departs from ρ .

KL Divergence Intuition

KL divergence measures how much information is lost when one probability ($\hat{\rho}_l$, the neuron's empirical firing rate) is used to approximate another (ρ , the desired tiny firing rate).

Back-Propagating the Sparsity Penalty

Derivative w.r.t. the Average Activation

Writing $-\Omega$ in expanded form and differentiating w.r.t. $\hat{\rho}_l$:

$$\frac{\partial \Omega(\theta)}{\partial \hat{\rho}_l} = -\frac{\rho}{\hat{\rho}_l} + \frac{1-\rho}{1-\hat{\rho}_l}.$$

Chain Rule to the Weights

For each hidden neuron l we need $\frac{\partial \Omega}{\partial W}$ via $\hat{\rho}_l \rightarrow h_l \rightarrow W$:

$$\frac{\partial \Omega(\theta)}{\partial W} = \frac{\partial \Omega(\theta)}{\partial \hat{\rho}_l} \frac{\partial \hat{\rho}_l}{\partial W}.$$

For a single weight W_{jl} , with $\hat{\rho}_l = \frac{1}{m} \sum_i g(\mathbf{w}_l^\top \mathbf{x}_i + b_l)$,

$$\frac{\partial \hat{\rho}_l}{\partial W_{jl}} = \frac{1}{m} \sum_{i=1}^m g'(\mathbf{w}_l^\top \mathbf{x}_i + b_l) x_{ij}, \quad \Rightarrow \quad \frac{\partial \hat{\rho}_l}{\partial \mathbf{w}_l} = \mathbf{x}_i (g'(\mathbf{w}_l^\top \mathbf{x}_i + b_l))^\top.$$

Final Gradient

$$\frac{\partial \widetilde{\mathcal{L}}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial W} + \frac{\partial \Omega(\theta)}{\partial W}.$$

Sparsity prevents the network from memorising: even with many hidden neurons, only a few are allowed to fire for any given input, so the model is forced to learn meaningful patterns.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders**
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

The Contractive Penalty

Goal

A contractive autoencoder also tries to prevent an overcomplete autoencoder from learning the identity function. It does so by adding a **Jacobian** penalty to the loss:

$$\Omega = \|\mathbf{J}_{\mathbf{x}}(\mathbf{h})\|_F^2 \quad (\text{Frobenius norm of the Jacobian}).$$

What the Jacobian Looks Like

If the input has n dimensions and the hidden layer has k dimensions, $\mathbf{J}_{\mathbf{x}}(\mathbf{h}) \in \mathbb{R}^{k \times n}$:

$$\mathbf{J}_{\mathbf{x}}(\mathbf{h}) = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \frac{\partial h_1}{\partial x_n} \\ \frac{\partial h_2}{\partial x_1} & \cdots & \frac{\partial h_2}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial h_k}{\partial x_1} & \cdots & \frac{\partial h_k}{\partial x_n} \end{bmatrix}$$

The (l, j) entry captures the variation in the output of the l -th neuron with a small variation in the j -th input.

The Penalty in Sum Form

$$\|\mathbf{J}_{\mathbf{x}}(\mathbf{h})\|_F^2 = \sum_{j=1}^n \sum_{l=1}^k \left(\frac{\partial h_l}{\partial x_j} \right)^2, \quad \min_{\theta} \mathcal{L}(\theta) + \Omega(\theta).$$

Intuition: Two Contradicting Objectives

What $\partial h_l / \partial x_j = 0$ Means

A small Jacobian entry $\partial h_l / \partial x_j \approx 0$ means the neuron is **not sensitive** to variations in input x_j . The contractive penalty pushes the hidden representation to be insensitive to input changes — it wants **h** to stay put if we perturb **x**.

The Apparent Contradiction

But the reconstruction loss $\mathcal{L}(\theta)$ *requires* **h** to capture variation in the input (otherwise it cannot reconstruct). So we have **two contradicting objectives**:

- $\mathcal{L}(\theta)$: *capture* important variation in the data,
- $\Omega(\theta)$: *do not capture* variation in the data.

Resolution: Capture Only Important Variation

By pitting these two objectives against each other, we ensure **h** is sensitive to *only the very important variations* observed in the training data, and insensitive to unimportant (noisy) directions.

Trade-off: capture only the very important variation in the data.

Contractive AE: The PCA Geometry

Sensitive Along High-Variance Directions Only

Consider the variation in the data along two directions $\mathbf{u}_1$ (high variance) and $\mathbf{u}_2$ (low variance, essentially noise):

- It makes sense to let a neuron be **sensitive** to variation along $\mathbf{u}_1$ (important for reconstruction).
- It makes sense to **inhibit** sensitivity to variation along $\mathbf{u}_2$ (small, noisy, unimportant).

By doing so we balance the contradicting goals of good reconstruction and low sensitivity.

Summary of the Regularised Autoencoders

- **PCA $\equiv$ linear AE:** $\min \|X - HW^*\|_F^2$, $H = XV_{:, \leq k}$, related to the SVD $X = U\Sigma V^\top$.
- **L_2 / weight decay:** $\Omega(\theta) = \lambda \|\theta\|^2$, tie weights $W^* = W^\top$.
- **Sparse:** $\Omega(\theta) = \sum_l \rho \log \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_l}$.
- **Denoising:** corrupt with $P(\tilde{x}_{ij} | x_{ij})$.
- **Contractive:** $\Omega(\theta) = \sum_{j,l} (\partial h_l / \partial x_j)^2$.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off**
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Setup: Fitting Two Models

The Experiment

We fit a curve to a set of points sampled from a true (unknown) sinusoidal function. We compare two models:

Simple: $y = \hat{f}(x) = w_1 x + w_0$ (degree 1), Complex: $y = \hat{f}(x) = \sum_{i=1}^{25} w_i x^i + w_0$ (degree 25).

In both cases we make an assumption about the form, with no idea of the true relation $f(x)$.

Repeated Sampling

We sample (say) 25 points from the training data and train a simple and a complex model. We repeat k times — each model is trained on a *different* sample of training data.

Observations

- **Simple models** trained on different samples do not differ much from each other (low spread), but all lie far from the true sinusoid $\Rightarrow$ **underfitting**.
- **Complex models** trained on different samples are very different from each other (high spread) $\Rightarrow$ **overfitting**; different data $\Rightarrow$ very different output.

Defining Bias and Variance

Bias

Let $f(x)$ be the true model and $\hat{f}(x)$ our estimate. The **bias** measures how far the *average* model is from the truth:

$$\text{Bias}(\hat{f}(x)) = \mathbb{E}[\hat{f}(x)] - f(x).$$

The simple model's average (blue line) is far from the true sinusoid $\Rightarrow$ **high bias**. The complex model's average is close $\Rightarrow$ **low bias**.

Variance

The **variance** measures how much the estimate spreads across different training samples:

$$\text{Var}(\hat{f}(x)) = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2].$$

Roughly, it tells us how much the different $\hat{f}(x)$'s (trained on different samples) differ from each other. Simple model: **low variance**; complex model: **high variance**.

Informal Summary

Simple model: **high bias, low variance**. Complex model: **low bias, high variance**. There is always a **trade-off**; both bias and variance contribute to the mean-squared error (MSE).

The Bias–Variance Decomposition

Statement

For a new point with $y = f(x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, the expected squared prediction error decomposes as

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}^2}_{\text{}} + \underbrace{\text{Variance}}_{\text{}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

Proof Sketch

Write $y - \hat{f} = (f - \mathbb{E}[\hat{f}]) + (\mathbb{E}[\hat{f}] - \hat{f}) + \varepsilon$. Expanding the square and taking expectations, the cross terms vanish because (i) $\mathbb{E}[\varepsilon] = 0$ and ε is independent of $\hat{f}$, and (ii)

$\mathbb{E}[\mathbb{E}[\hat{f}] - \hat{f}] = 0$. Using

$$A = f - \mathbb{E}[\hat{f}], \quad B = \varepsilon, \quad C = \mathbb{E}[\hat{f}] - \hat{f},$$

only $\mathbb{E}[A^2] = \text{Bias}^2$, $\mathbb{E}[B^2] = \sigma^2$, $\mathbb{E}[C^2] = \text{Variance}$ survive.

Conclusion

Expected error = Bias² + Variance + Irreducible Noise.

Reducing bias (more complexity) increases variance and vice versa — we must balance the two.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error**
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

The Two Errors

Train Error and Test Error

The parameters of $\hat{f}(x)$ are trained on a training set $\{(x_i, y_i)\}_{i=1}^n$. At test time we evaluate on a *validation/unseen* set not used for training. This gives two quantities:

$$\text{train}_{\text{err}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2, \quad \text{test}_{\text{err}} = \frac{1}{m} \sum_{i=n+1}^{n+m} (y_i - \hat{f}(x_i))^2.$$

As model complexity rises, train error becomes overly optimistic and gives a wrong picture of how close $\hat{f}$ is to f . The validation error gives the real picture.

True Error

For an unseen point we want the **true error** $\mathbb{E}[(\hat{f}(x_i) - f(x_i))^2]$. We cannot estimate it directly because we do not know f . We estimate it empirically using observations y_i and predictions $\hat{y}_i$.

Model Used

$D = \{(x_i, y_i)\}_{i=1}^{m+n}$ with $y_i = f(x_i) + \varepsilon_i$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, and $\hat{f}$ trained on $T \subset D$ (training data), giving predictions $\hat{y}_i = \hat{f}(x_i)$.

Relating Empirical Error to True Error

Empirical Expectation

The empirical squared error over observations is

$$\mathbb{E}[(\hat{y}_i - y_i)^2] = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2.$$

(Analogy: to estimate expected goals $\mathbb{E}[\text{goals}]$ over k matches we use $\frac{1}{k} \sum_i g_i$.)

Substituting $y_i = f(x_i) + \varepsilon_i$

Expanding $\mathbb{E}[(\hat{y}_i - y_i)^2]$ with $\hat{y}_i = \hat{f}(x_i)$:

$$\mathbb{E}[(\hat{f}(x_i) - f(x_i))^2] = \mathbb{E}[(\hat{y}_i - y_i)^2] - \mathbb{E}[\varepsilon_i^2] + 2\mathbb{E}[\varepsilon_i(\hat{f}(x_i) - f(x_i))].$$

We can empirically evaluate the first RHS term using observations. The last term is a **covariance** between ε_i and $\hat{f}(x_i) - f(x_i)$ (since $\mathbb{E}[\varepsilon_i] = 0$).

Covariance Reminder

$\text{Cov}(x, y) = \mathbb{E}[(x - \mu_x)(y - \mu_y)] = \mathbb{E}[xy] - \mu_x \mu_y$; with $\mu_\varepsilon = 0$ the cross term is exactly $\mathbb{E}[\varepsilon_i(\hat{f}(x_i) - f(x_i))]$.

Case 1 — Using Test Observations

The Key Independence

With **test** data, none of the test observations participated in estimating $\hat{f}$ (its parameters were learned only on training data). Therefore the test noise ε_i is **independent** of $\hat{f}(x_i) - f(x_i)$:

$$\varepsilon_i \perp (\hat{f}(x_i) - f(x_i)).$$

The Cross Term Vanishes

For independent variables $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$, so

$$\mathbb{E}[\varepsilon_i(\hat{f}(x_i) - f(x_i))] = \underbrace{\mathbb{E}[\varepsilon_i]}_{=0} \mathbb{E}[\hat{f}(x_i) - f(x_i)] = 0.$$

Result

true error = empirical test error + small constant

i.e. $\mathbb{E}[(\hat{f}(x_i) - f(x_i))^2] = \frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2 - \frac{1}{m} \sum \varepsilon_i^2$. The test error is close to the true error, differing only by a small constant. That is why we use a validation set.

Case 2 — Using Training Observations (Optimism)

No Independence Here

With **training** data, ε_i was used to estimate the parameters of $\hat{f}(x)$. Hence ε_i is correlated with $\hat{f}(x_i)$, and

$$\mathbb{E}[\varepsilon_i (\hat{f}(x_i) - f(x_i))] \neq \mathbb{E}[\varepsilon_i] \mathbb{E}[\hat{f}(x_i) - f(x_i)] \neq 0.$$

Consequence: Optimism

The covariance term is positive, so the empirical **train error is smaller** than the true error — it does not give the true picture of the error. The training error is **overly optimistic**.

Practical Rule

Even gradient-based optimisers (SGD, Adam) make stationary distribution assumptions about the noise; if those hold, the noise is zero-mean and the analysis carries through. Regardless,

Always use a validation set (independent of training) to estimate error.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity**
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)

Quantifying the Optimism via Stein's Lemma

Stein's Lemma Result

Using Stein's lemma (with some algebra) one can show the train-error covariance term equals

$$\frac{1}{n} \sum_{i=1}^n \varepsilon_i (\hat{f}(x_i) - f(x_i)) = \frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$$

(Stirling's approximation $n! \approx \sqrt{2\pi n}(n/e)^n$ is used in the derivation of related results.)

When is $\partial \hat{f}(x_i) / \partial y_i$ Large?

It is large when a *small* change in the observation y_i causes a *large* change in the estimate $\hat{f}$. A **complex** model is more sensitive to changes in the observations; a **simple** model is less sensitive. So this derivative directly measures model complexity.

Verification

Fit a simple and a complex model to data, then change one data point and retrain. The simple model barely changes; the complex model changes a lot — confirming complex models are more sensitive.

The Master Equation of Regularisation

True Error = Train Error + Complexity

Combining the pieces:

$$\text{true error} = \text{empirical train error} + \text{small constant} + \Omega(\text{model complexity})$$

where $\Omega(\theta)$ approximates $\frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial^2 \hat{f}(x_i)}{\partial y_i}$; it is high for complex models, small for simple models.

What We Should Actually Minimise

Instead of minimising the training error $\mathcal{L}_{\text{train}}(\theta)$ alone, we should minimise

$$\min_{\theta} \underbrace{\mathcal{L}_{\text{train}}(\theta) + \Omega(\theta)}_{\mathcal{L}(\theta)}.$$

$\Omega(\theta)$ should ensure the model has *reasonable* complexity, so the model generalises best on test data. This is the **basis for all regularisation methods**.

Why We Care (Universal Approximation)

By the Universal Approximation Theorem, deep neural networks are highly complex models with many parameters and non-linearities; it is easy for them to overfit and drive training error to zero. Hence we need some form of regularisation.

The Sweet Spot

Train Error vs Test Error vs Complexity

As model complexity increases:

- **Train error** decreases monotonically (towards 0).
- **Test error** first decreases (less bias) then increases (more variance) — a U-shape.

The minimum of the test-error curve is the **sweet spot**: the ideal model complexity, achieved by tuning $\Omega(\theta)$.

Different Forms of Regularisation

All of the following are instances of adding an $\Omega(\theta)$ that controls complexity:

- L_2 regularisation (weight decay)
- Dataset augmentation
- Parameter sharing and tying
- Adding noise to the inputs
- Adding noise to the outputs (label smoothing)
- Early stopping
- Ensemble methods
- Dropout

We treat each in turn.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation**
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)

L_2 Regularisation: Setup

Regularised Loss

$$\widetilde{\mathcal{L}}(\mathbf{w}) = \underbrace{\mathcal{L}(\mathbf{w})}_{\text{empirical train error}} + \frac{\alpha}{2} \|\mathbf{w}\|^2.$$

The penalty relates to model complexity by *reducing the freedom* of the weights.

Gradient and Update

$$\nabla \widetilde{\mathcal{L}}(\mathbf{w}) = \nabla \mathcal{L}(\mathbf{w}) + \alpha \mathbf{w}, \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \mathcal{L}(\mathbf{w}_t) - \eta \alpha \mathbf{w}_t.$$

This requires only a very small modification to the code: subtract an extra $\eta \alpha \mathbf{w}_t$ each step (weight decay).

Taylor Expansion around the Unregularised Optimum

Let $\mathbf{w}^*$ be optimal for the *unregularised* loss, so $\nabla \mathcal{L}(\mathbf{w}^*) = 0$. Expanding to second order with Hessian H :

$$\mathcal{L}(\mathbf{w}) \approx \mathcal{L}(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^\top H(\mathbf{w} - \mathbf{w}^*), \quad \nabla \mathcal{L}(\mathbf{w}) = H(\mathbf{w} - \mathbf{w}^*).$$

L_2 : Solving for the Regularised Optimum

Set the Regularised Gradient to Zero

Let $\tilde{\mathbf{w}}$ be optimal for $\tilde{\mathcal{L}}$, so $\nabla \tilde{\mathcal{L}}(\tilde{\mathbf{w}}) = 0$:

$$\nabla \mathcal{L}(\tilde{\mathbf{w}}) + \alpha \tilde{\mathbf{w}} = H(\tilde{\mathbf{w}} - \mathbf{w}^*) + \alpha \tilde{\mathbf{w}} = 0 \Rightarrow (H + \alpha I)\tilde{\mathbf{w}} = H\mathbf{w}^* \Rightarrow \tilde{\mathbf{w}} = (H + \alpha I)^{-1} H\mathbf{w}^*$$

As $\alpha \rightarrow 0$, $\tilde{\mathbf{w}} \rightarrow \mathbf{w}^*$ (no regularisation). We analyse $\alpha \neq 0$.

Diagonalise the Hessian

Assume H is symmetric PSD with $H = Q\Lambda Q^\top$ (Q orthogonal, $QQ^\top = Q^\top Q = I$). Then

$$\begin{aligned}\tilde{\mathbf{w}} &= (Q\Lambda Q^\top + \alpha QIQ^\top)^{-1} Q\Lambda Q^\top \mathbf{w}^* = (Q(\Lambda + \alpha I)Q^\top)^{-1} Q\Lambda Q^\top \mathbf{w}^* \\ &= Q(\Lambda + \alpha I)^{-1} Q^\top Q\Lambda Q^\top \mathbf{w}^* = Q \underbrace{(\Lambda + \alpha I)^{-1} \Lambda}_D Q^\top \mathbf{w}^*.\end{aligned}$$

Result

$$\tilde{\mathbf{w}} = Q(\Lambda + \alpha I)^{-1} \Lambda Q^\top \mathbf{w}^*, \quad D = (\Lambda + \alpha I)^{-1} \Lambda \text{ is diagonal.}$$

L_2 : What Is Happening Geometrically

Rotate, Scale, Rotate Back

The operation $\tilde{\mathbf{w}} = QDQ^\top \mathbf{w}^*$ does three things:

- 1 $Q^\top \mathbf{w}^*$: rotate $\mathbf{w}^*$ into the eigenbasis.
- 2 D : scale each coordinate.
- 3 Q : rotate back.

If $\alpha = 0$ then $D = I$ and Q rotates $Q^\top \mathbf{w}^*$ back to $\mathbf{w}^*$ (nothing changes).

The Diagonal Scaling

For $\alpha \neq 0$, $D = \text{diag}\left(\frac{d_i}{d_i + \alpha}\right)$ where d_i are the eigenvalues of H :

$$d_i \gg \alpha \Rightarrow \frac{d_i}{d_i + \alpha} \approx 1 \quad (\text{direction kept}), \quad d_i \ll \alpha \Rightarrow \frac{d_i}{d_i + \alpha} \approx 0 \quad (\text{direction killed}).$$

Interpretation

The weight vector is rotated to $\tilde{\mathbf{w}}$; *all* elements shrink, but some shrink *more* than others. Important directions (large eigenvalue, large loss curvature) are retained; unimportant directions are scaled down. This is exactly **dimension-specific** shrinkage — what we wanted. The effective number of parameters is $\sum_{i=1}^n \frac{d_i}{d_i + \alpha} \ll n$.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation**
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)

Dataset Augmentation

Idea

Minimising the empirical train error on a small dataset leads to perfect classification of training points (overfitting). Augmentation creates *more* labelled data using domain knowledge of the task. Given an image, we exploit the fact that certain transformations do **not change the label**:

- rotate by 20° , by 62° ,
- shift vertically / horizontally,
- blur,
- change some pixels.

Why It Works

Typically **more data** $\Rightarrow$ **better learning**. The augmented data is supervised data created using knowledge of the task. It works very well for image classification / object recognition and has also been shown to work for speech.

Caveat

For some tasks it may not be clear how to generate such label-preserving data (e.g. some NLP tasks), where naive transformations could change meaning. Augmentation must respect the semantics of the task.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying**
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Parameter Sharing vs. Parameter Tying

Parameter Sharing

- Used in **CNNs**.
- The same filter is applied at different positions of the image.
- Equivalently, the same weight matrix acts on different input neurons.
- Use the same parameter in multiple computations.

Parameter Tying

- Typically used in **autoencoders**.
- Encoder and decoder weights are tied.
- Impose an equality constraint between parameters: $W^* = W^\top$.

Why Tying Regularises

Tying is a specific form of sharing that **reduces the number of free parameters** and acts as regularisation (it reduces model complexity $\Omega(\theta)$, consistent with the Stein's-lemma view). With $W \in \mathbb{R}^{k \times d}$ and $W^* \in \mathbb{R}^{d \times k}$, tying $W^* = W^\top$ takes the parameter count from $2dk$ down to dk — **half the parameters**.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs**
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Adding Gaussian Noise to Inputs $\equiv$ Weight Decay

Setup

We already saw input noise in denoising autoencoders. For a simple linear input–output network, adding Gaussian noise to the input is equivalent to **weight decay** (L_2 regularisation). It can also be viewed as data augmentation. Let $\tilde{x}_i = x_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$.

Original vs. Noisy Output

$$\hat{y} = \sum_{i=1}^n w_i x_i \quad (\text{original}), \quad \tilde{y} = \sum_{i=1}^n w_i \tilde{x}_i = \hat{y} + \sum_{i=1}^n w_i \varepsilon_i.$$

Expected Squared Error with Noise

$$\begin{aligned} \mathbb{E}[(\tilde{y} - y)^2] &= \mathbb{E}\left[\left((\hat{y} - y) + \sum_i w_i \varepsilon_i\right)^2\right] \\ &= \mathbb{E}[(\hat{y} - y)^2] + 2\mathbb{E}\left[(\hat{y} - y) \sum_i w_i \varepsilon_i\right] + \mathbb{E}\left[\left(\sum_i w_i \varepsilon_i\right)^2\right]. \end{aligned}$$

Input Noise: The Cross Term Vanishes

Cross Term

Since $\mathbb{E}[\varepsilon_i] = 0$ and ε_i is independent of $\hat{y} - y$,

$$2\mathbb{E}\left[(\hat{y} - y) \sum_i w_i \varepsilon_i\right] = 2 \sum_i w_i \mathbb{E}[\hat{y} - y] \mathbb{E}[\varepsilon_i] = 0.$$

Noise Term

Using $\mathbb{E}[\varepsilon_i \varepsilon_j] = 0$ for $i \neq j$ (independent) and $\mathbb{E}[\varepsilon_i^2] = \sigma^2$,

$$\mathbb{E}\left[\left(\sum_i w_i \varepsilon_i\right)^2\right] = \sum_i \sum_j w_i w_j \mathbb{E}[\varepsilon_i \varepsilon_j] = \sigma^2 \sum_{i=1}^n w_i^2.$$

Result

$$\mathbb{E}[(\tilde{y} - y)^2] = \mathbb{E}[(\hat{y} - y)^2] + \sigma^2 \sum_{i=1}^n w_i^2$$

The extra term $\sigma^2 \sum_i w_i^2$ is exactly the L_2 norm penalty. Hence **adding noise to the input is the same as L_2 regularisation**.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)**
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Hard Targets vs. Soft Targets

Hard Targets

With one-hot labels, we minimise $\sum_{l \geq 0} p_l \log q_l$ where the true distribution is $p = \{0, 1, 0, \dots, 0\}$ (all mass on the correct class) and q is the estimate. Trusting these hard labels completely leads the model to drive training error to zero — overfitting.

Soft Targets (Label Smoothing)

Do not trust the labels as perfectly true — they may be noisy. Instead use **soft targets**: spread a small amount of probability mass ε across the wrong classes,

$$p = \left\{ \frac{\varepsilon}{k-1}, \frac{\varepsilon}{k-1}, \dots, 1 - \varepsilon, \dots, \frac{\varepsilon}{k-1} \right\},$$

where ε is the noise in the label (a small positive constant) and k is the number of classes.

Why It Acts Like Regularisation

The aim of regularisation is to not overfit the training data. Soft targets ensure the model treats the labels as only *partly* reliable, giving corrupted information. The principle: **do not allow the true training error (computed from training data) to go to zero** — do whatever you can to avoid that. This is a heuristic.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping**
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Early Stopping: The Algorithm

Procedure

- Track the **validation error** as training proceeds.
- Keep a **patience** parameter p .
- Every p steps, check whether validation error has improved (got worse, increased, or remained the same).
- If at step k there was *no improvement* in the previous p steps, **stop** training and return the model stored at step $k - p$.

Training error keeps decreasing, but validation error follows a U-shape — we stop near its minimum.

Example (patience $p = 5$)

Epoch	Val. loss
10	0.25
11	0.23
12	0.24
13	0.25
14	0.25
15	0.24
16	0.24

Properties

Very effective and one of the most widely used forms of regularisation. Can be combined with other regularisers (such as L_2). But *how* does it act as a regulariser, given gradient descent only sees training data?

Early Stopping: Why It Limits Weight Growth

Bounding the Update

The SGD update $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \mathcal{L}(\mathbf{w}_t)$ unrolls to $\mathbf{w}_{t+1} = \mathbf{w}_0 - \eta \sum_{k=1}^t \nabla \mathcal{L}(\mathbf{w}_k)$. If τ is the maximum value of $\|\nabla \mathcal{L}\|$, then

$$\|\mathbf{w}_{t+1}\| \leq \|\mathbf{w}_0\| + \eta t \tau.$$

So t controls how far $\mathbf{w}_t$ can travel from $\mathbf{w}_0$ — limited steps means limited weight growth.

Connecting to the Hessian

Using the Taylor expansion $\nabla \mathcal{L}(\mathbf{w}) = H(\mathbf{w} - \mathbf{w}^*)$ and starting from $\mathbf{w}_0 = 0$, the SGD update becomes

$$\mathbf{w}_t = (I + \eta H) \mathbf{w}_{t-1} - \eta H \mathbf{w}^*,$$

and with $H = Q \Lambda Q^\top$ one can show

$$\mathbf{w}_t = Q (I - (I - \varepsilon \Lambda)^t) Q^\top \mathbf{w}^* \quad (\text{rotation} \rightarrow \text{scaling} \rightarrow \text{rotation}).$$

Comparison with L_2

Recall the L_2 optimum $\tilde{\mathbf{w}} = Q (I - (\Lambda + \alpha I)^{-1} \alpha) Q^\top \mathbf{w}^*$. The two coincide (i.e. $\mathbf{w}_t = \tilde{\mathbf{w}}$) when ε , t and α are chosen so that $(I - \varepsilon \Lambda)^t = (I + \alpha \Lambda^{-1})^{-1}$. So early stopping is, in effect, L_2 regularisation in disguise.

Early Stopping: Direction-Wise View

Important vs. Unimportant Directions

- Early stopping only allows t updates to the parameters.
- If a parameter / direction $\mathbf{w}$ is **important** for the loss $\mathcal{L}(\theta)$, then $\partial \mathcal{L} / \partial \mathbf{w}$ is large, so its updates are large and it grows quickly.
- If a parameter is **not important**, $\partial \mathcal{L} / \partial \mathbf{w}$ is small, its updates are small, and within only t steps it cannot grow large.

Conclusion

By doing a *limited* number of steps t , early stopping keeps unimportant weights small (just like L_2 shrinks them via $d_i / (d_i + \alpha)$), retaining only the significant directions. This is why early stopping generalises.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)**

Ensembles and Bagging

Idea

Combine the outputs of different models to reduce generalisation error. The models can respond to different classifiers, or be different instances of the same classifier trained with: different hyperparameters, different features, or different samples of training data.

Bagging

Form an ensemble of different instances of the *same* classifier. From a given dataset, construct multiple training sets $T_1, T_2, \dots, T_k$ by **sampling with replacement** (each a subset). Train one model on each.

When Would Bagging Work?

Each classifier makes certain errors on different points. We *want the errors to be as independent as possible* across models — then averaging cancels them out.

Bagging: The MSE of the Ensemble

Setup

Consider k models. Model i makes error ε_i on a test example, where each ε_i is drawn from a zero-mean multivariate normal with

$$\text{Var} = \mathbb{E}[\varepsilon_i^2] = V, \quad \text{Cov} = \mathbb{E}[\varepsilon_i \varepsilon_j] = C \ (i \neq j).$$

The average prediction error is $\frac{1}{k} \sum_i \varepsilon_i$.

Expected Squared Error

$$\begin{aligned} \text{mse} &= \mathbb{E} \left[\left(\frac{1}{k} \sum_i \varepsilon_i \right)^2 \right] = \frac{1}{k^2} \mathbb{E} \left[\sum_i \varepsilon_i^2 + \sum_i \sum_{j \neq i} \varepsilon_i \varepsilon_j \right] \\ &= \frac{1}{k^2} \left[\sum_i \mathbb{E}[\varepsilon_i^2] + \sum_i \sum_{j \neq i} \mathbb{E}[\varepsilon_i \varepsilon_j] \right] = \frac{1}{k^2} [kV + k(k-1)C] \\ &= \boxed{\frac{1}{k} V + \frac{k-1}{k} C} \end{aligned}$$

(using k terms with variance V and $k(k-1)$ cross terms with covariance C).

Bagging: When Does It Help?

Two Extremes

- **Errors perfectly correlated** ($C = V$):

$$\text{mse} = \frac{1}{k} V + \frac{k-1}{k} V = V.$$

Bagging **does not help**: the ensemble MSE is as bad as a single model.

- **Errors independent / uncorrelated** ($C = 0$):

$$\text{mse} = \frac{1}{k} V.$$

The ensemble MSE **shrinks by a factor of k** .

Takeaway

Bagging works precisely when the models' errors are independent (or uncorrelated): $V \rightarrow V/k$. The more independent the errors, the larger the gain. This is why we want diverse models (different samples, features, hyperparameters).

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Dropout: Motivation

The Cost Problem with Ensembles

Model averaging (a bagging ensemble) almost always helps, but training several large neural networks for an ensemble is prohibitively expensive — whether we train networks with different architectures (option 1) or multiple instances of the same network on different samples (option 2). Combining several models at *test* time is also infeasible for real-time applications.

What Dropout Does

Dropout addresses both issues. It effectively allows training of exponentially many neural networks without significant computational overhead, and gives an efficient approximate way of *combining* them.

Mechanism

Dropout means **dropping out units**: temporarily remove a node and its incoming/outgoing connections, producing a *thinned* network. Each node is retained with a fixed probability p (typically $p = 0.5$ for hidden nodes and $p = 0.8$ for visible/input nodes).

Dropout: Exponentially Many Thinned Networks

Counting the Networks

With n nodes, each can be retained or dropped, giving

$$2^n$$

possible thinned networks. This is prohibitively large — we cannot train them all separately.

The Two Tricks

- 1 **Share the weights** across all thinned networks (one master weight matrix W).
- 2 **Sample a different thinned network for each training instance** (or mini-batch).

Training Loop

Initialise weights. For training instance / mini-batch 1, apply dropout to get a thinned network, compute the loss, back-propagate — updating *only the active weights*. For instance 2, apply dropout again to get a *different* thinned network, and repeat. Because of weight sharing, the same master weights get updated through many different thinned networks over time.

Dropout: Parameter Sharing Across Networks

Updates per Weight

- If a weight was active in *both* instances, by now it has received *two* updates.
- If active in only *one*, it has received *one* update.

Each thinned network is trained rarely (or even never) on its own, but parameter sharing ensures **no weight is left untrained**: the shared W is updated many times across the various thinned networks that contain it.

What Happens at Test Time?

We have 2^n thinned networks; at test time each would have been used with probability p . It is impossible to aggregate the outputs of 2^n networks. Instead, we use the **full network** and *scale* the output of each node by the fraction of time it was on during training (i.e. by p). A node always present at test time but on only a p -fraction of training time has its outgoing activation multiplied by p .

Dropout: Preventing Co-adaptation

Masking Noise on Hidden Units

Dropout essentially applies a **masking noise** to the hidden units. It prevents hidden units from *co-adapting*: a hidden unit cannot rely too much on other specific units, since any of them may be dropped at any time.

Robustness Through Redundancy

Suppose hidden unit h_i learns to detect a face by relying on another unit that detects the nose. If that nose-detector is dropped, h_i fails. Dropping a unit corresponds to **erasing the information** it encodes. To cope, the model must learn another unit that *redundantly* encodes the nose, or learn to detect the face using *other* features.

Intuition

Like discouraging lazy team members who depend on others: dropout forces every hidden unit to be useful on its own, so each learns to be more robust.

“*Koru ekule enkoru leru*” — no free-riding; each unit must pull its weight.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

Gradient Descent and the Chain Rule

Single Neuron

For a one-neuron network $\hat{y} = \sigma(wx)$ with loss $\mathcal{L}$,

$$w = w - \eta \nabla w, \quad \nabla w = \frac{\partial \mathcal{L}(w)}{\partial w} = (f(x) - y) f(x)(1 - f(x)) x.$$

Wider Network

With several inputs x_1, x_2, x_3 and weights w_1, w_2, w_3 ,

$$w_j = w_j - \eta \frac{\partial \mathcal{L}}{\partial w_j}, \quad \nabla w_j = (f(x) - y) f(x)(1 - f(x)) x_j.$$

Deeper Network (Back-Propagation)

With $h_2 = \sigma(a_2)$, $a_2 = w_2 h_1$, $h_1 = \sigma(a_1)$, $a_1 = w_1 x = w_1 h_0$, the chain rule gives

$$\nabla w_1 = \frac{\partial \mathcal{L}(w)}{\partial w_1} = \frac{\partial \mathcal{L}(w)}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_2} \frac{\partial a_2}{\partial h_1} \frac{\partial h_1}{\partial a_1} \frac{\partial a_1}{\partial w_1} = (\dots) h_0.$$

The Two Key Observations

Gradient $\propto$ Input

In general $\nabla w_i = \frac{\partial \mathcal{L}(w)}{\partial w_i} = (\dots) h_{i-1}$: the gradient w.r.t. a parameter is **proportional to the input feeding into that parameter** (h_{i-1} or, for the first layer, x_i). It directly depends on the activation of the preceding layer.

Deep and Wide Networks

For a deep, wide network, ∇w_2 is given by the chain rule applied across **multiple paths** — this is exactly back-propagation. The gradients tell us the *responsibility* of each parameter towards the loss, and they always have the same shape as the variable being differentiated.

Historical Note

Back-propagation was popularised by Rumelhart *et al.* in 1986. It did not catch on immediately due to limited **computational power** and **memory**, and was not very successful for really deep networks. In fact, until 2006 it was very hard to train very deep networks — even after many epochs the training often did not converge.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)
- 18 Dropout

The Discrete Cosine Transform (DCT)

Objective

Interpret the 2D DCT: represent an image as a **sum of basis images**. Compression then amounts to setting some DCT coefficients to zero — this is the heart of **JPEG**.

Forward 2D DCT

$$F(u, v) = \frac{2}{N} C(u)C(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right)$$

with $C(k) = \frac{1}{\sqrt{2}}$ if $k = 0$ and $C(k) = 1$ otherwise. Here $f(x, y)$ is the pixel value, $F(u, v)$ a DCT coefficient, N the image size, (x, y) spatial coordinates, (u, v) frequency coordinates.

Image as a Weighted Sum of Basis Images

$$X(m, n) = \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} b_{k,l} C_{k,l}[m, n],$$

where the weights $b_{k,l}$ are the discrete cosine coefficients and each basis image $C_{k,l}[m, n]$ is the product of a vertical frequency $\frac{k}{2N}$ cosine and a horizontal frequency $\frac{l}{2N}$ cosine.

Worked DCT Example (2×2 Image)

Setup

Let $f(x, y) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $N = 2$, with $\cos(0) = 1$, $\cos(\pi/4) = 0.707$, $\cos(3\pi/4) = -0.707$.

Compute $F(0, 0)$ ($u = 0, v = 0$)

$$F(0, 0) = \frac{2}{2} C(0) C(0) \sum_{x=0}^1 \sum_{y=0}^1 f(x, y) \cdot 1 = \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} (1 + 2 + 3 + 4) = \frac{1}{2} \cdot 10 = 5.$$

The DC coefficient $F(0, 0)$ is proportional to the average pixel value.

Other Coefficients

Carrying out the analogous sums gives, for this image, $F(0, 1) \approx -1$, $F(1, 0) \approx -2$, $F(1, 1) = 0$, so the DCT matrix is

$$F = \begin{bmatrix} 5 & -1 \\ -2 & 0 \end{bmatrix}.$$

A horizontal edge in the image becomes energy concentrated in the first column of the DCT matrix, because the image varies only in the vertical direction and has no horizontal frequency content.

Compression and the JPEG Pipeline

Compression by Dropping Coefficients

Storing all coefficients reproduces the image exactly (no loss). Keeping only the few *significant* coefficients (e.g. 4 out of many) compresses with little visible loss, because most of the energy lives in a handful of low-frequency coefficients.

JPEG Encoding Steps

- 1 Colour-space conversion: $RGB \rightarrow YC_bC_r$ (luminance Y , blue chrominance C_b , red chrominance C_r). Reversible, no data compression.
- 2 Chrominance down-sampling (2×2 averaging of C_b, C_r): removes a considerable amount of data the eye barely notices.
- 3 Break the $M \times N$ image into 8×8 blocks; take the 2D DCT of each block.
- 4 **Quantisation**: divide each coefficient by a quantisation table and round to the nearest integer (this is where data is actually thrown away).
- 5 Run-length encoding (RLE) and Huffman encoding of the quantised coefficients.

JPEG Decoding

Read the bits for each 8×8 block, convert bits to DCT coefficients, take the inverse DCT to recover the 8×8 image, and put the blocks back together.

Why JPEG Works: Exploiting the Human Eye

Eyes Are Not Perfect

The human eye has ~ 100 million rod cells (brightness, low-light sensitive) but only ~ 6 million cone cells (colour sensitive). So we perceive **brightness (luminance)** far better than **colour (chrominance)**. JPEG removes information we do not perceive well.

DCT + Quantisation Together

The DCT alone does not compress; quantisation does. The **quantisation table** uses smaller divisors for low-frequency (top-left) coefficients — which the eye notices — and larger divisors for high-frequency (bottom-right) details the eye cannot perceive easily. After rounding, the high-frequency details are thrown away. We literally cannot tell the difference.

High-Frequency Blindness

Eyes are good at seeing edges (e.g. the outline of a rock) but poor at high-frequency colour detail (single blades of grass, individual leaves in a cluster, shadow variation). The chrominance quantisation table uses higher numbers (coarser) than the luminance table. Then **run-length** encoding compactly stores the many resulting zeros (e.g. “ 0×38 ”), and **Huffman** coding compresses the rest. H.264 (AVC, Advanced Video Coding) extends these ideas to video.

Current Section

- 1 Introduction to Autoencoders
- 2 Choice of Decoder and Loss Function
- 3 Link between PCA and Autoencoders
- 4 Regularisation in Autoencoders
- 5 Denoising Autoencoders
- 6 Sparse Autoencoders
- 7 Contractive Autoencoders
- 8 The Bias–Variance Trade-off
- 9 Training Error vs Test Error
- 10 True Error and Model Complexity
- 11 L_2 Regularisation
- 12 Dataset Augmentation
- 13 Parameter Sharing and Tying
- 14 Adding Noise to the Inputs
- 15 Adding Noise to the Outputs (Label Smoothing)
- 16 Early Stopping
- 17 Ensemble Methods (Bagging)

18 Dropout

Summary: The Big Picture

Autoencoders

Encode $\mathbf{h} = g(W\mathbf{x} + \mathbf{b})$, decode

$\hat{\mathbf{x}} = f(W^*\mathbf{h} + \mathbf{c})$.

Undercomplete: compresses (like PCA).

Overcomplete: can learn identity $\Rightarrow$ needs regularisation.

Binary input: sigmoid + cross-entropy.

Real input: linear + squared error.

PCA $\equiv$ Linear AE

With linear encoder/decoder, squared loss, and $\frac{1}{\sqrt{m}}$ centring: $H = XV_{\cdot, \leq k}$, and the encoder $W = V_{\cdot, \leq k}$ equals the PCA projection P (eigenvectors of $X^T X$).

Generalisation Theory

Expected error = Bias² + Variance + noise.

true error = train error + const

+ $\Omega(\text{complexity})$,

$\Omega \approx \frac{\sigma^2}{n} \sum_i \partial \hat{f}(x_i) / \partial y_i$ (Stein's lemma).

Minimise $\mathcal{L}_{\text{train}}(\theta) + \Omega(\theta)$.

Regularisers

L_2 scales weights by $\frac{d_i}{d_i + \alpha}$; denoising; sparse (KL); contractive (Jacobian); weight tying; augmentation; input noise $\equiv L_2$; label smoothing; early stopping $\equiv L_2$; bagging ($V \rightarrow V/k$); dropout (2^n shared networks).

“An autoencoder learns a representation; regularisation stops it from cheating. Every regulariser is really one principle — do not let the training error collapse to zero.”

Thank You

Questions?